

Reflection for C++26

Document #: P2996R0
Date: 2023-10-15
Project: Programming Language C++
Audience: EWG
Reply-to: Wyatt Childers
<wcc@edg.com>
Peter Dimov
<pdimov@gmail.com>
Barry Revzin
<barry.revzin@gmail.com>
Andrew Sutton
<andrew.n.sutton@gmail.com>
Faisal Vali
<faisalv@gmail.com>
Daveed Vandevoroorde
<daveed@edg.com>

Contents

[1 Introduction](#)

[1.1 Notable Additions to P1240](#)

[1.2 Why a single opaque reflection type?](#)

[2 Examples](#)

[2.1 Back-And-Forth](#)

[2.2 Selecting Members](#)

[2.3 List of Types to List of Sizes](#)

[2.4 Implementing `make\_integer\_sequence`](#)

[2.5 Getting Class Layout](#)

[2.6 Enum to String](#)

[2.7 Parsing Command-Line Options](#)

[2.8 A Simple Tuple Type](#)

[2.9 Struct to Struct of Arrays](#)

[2.10 Parsing Command-Line Options II](#)

[2.11 A Universal Formatter](#)

[2.12 Implementing `member-wise hash\_append`](#)

[2.13 Converting a Struct to a Tuple](#)

[3 Proposed Features](#)

[3.1 The Reflection Operator \(`^`\)](#)

[3.2 Splicers \(`\[ :...: \]`\)](#)

[3.2.1 Range Splicers](#)

[3.3 `std::meta::info`](#)

[3.4 Metafunctions](#)

[3.4.1 invalid reflection, is_invalid, diagnose_error](#)

[3.4.2 name of, display_name of, source_location of](#)

[3.4.3 type of, parent of, entity of](#)

[3.4.4 template of, template arguments of](#)

[3.4.5 members of, nonstatic data members of, bases of, enumerators of, subobjects of](#)

[3.4.6 substitute](#)

[3.4.7 entity_ref<T>, value_of<T>, ptr_to_member<T>](#)

[3.4.8 test_type<Pred>](#)

[3.4.9 Other Singular Reflection Predicates](#)

[3.4.10 reflect_value](#)

[3.4.11 nsdm_description, synth_struct, synth_union](#)

[3.4.12 Data Layout Reflection](#)

[4 References](#)

§ 1 Introduction

This is a proposal for a reduced initial set of features to support static reflection in C++. Specifically, we are mostly proposing a subset of features suggested in [\[P1240R2\]](#):

- the representation of program elements via constant-expressions producing *reflection values* — *reflections* for short — of an opaque type `std::meta::info`,
- a *reflection operator* (prefix `^`) that produces a reflection value for its operand construct,
- a number of `constexpr` *metafunctions* to work with reflections (including deriving other reflections), and
- constructs called *splicers* to produce grammatical elements from reflections (e.g., `[ : refl : ]`).

This proposal is not intended to be the end-game as far as reflection and compile-time metaprogramming are concerned. Instead, we expect it will be a useful core around which more powerful features will be added incrementally over time. In particular, we believe that most or all the remaining features explored in P1240R2 and that code injection (along the lines described in [\[P2237R0\]](#)) are desirable directions to pursue.

Our choice to start with something smaller is primarily motivated by the belief that that improves the chances of these facilities making it into the language sooner rather than later.

§ 1.1 Notable Additions to P1240

While we tried to select a useful subset of the P1240 features, we also made a few additions and changes. Most of those changes are minor. For example, we added a `std::meta::test_type` interface that makes it convenient to use existing standard type predicates (such as `is_class_v`) in reflection computations.

One addition does stand out, however: We have added metafunctions that permit the synthesis of simple struct and union types. While it is not nearly as powerful as generalized code injection (see [\[P2237R0\]](#)), it can be remarkably effective in practice.

§ 1.2 Why a single opaque reflection type?

Perhaps the most common suggestion made regarding the framework outlined in P1240 is to switch from the single `std::meta::info` type to a family of types covering various language elements (e.g., `std::meta::variable`, `std::meta::type`, etc.).

We believe that doing so would be mistake with very serious consequences for the future of C++.

Specifically, it would codify the language design into the type system. We know from experience that it has been quasi-impossible to change the semantics of standard types once they were standardized, and there is no reason to think that such evolution would become easier in the future. Suppose for example that we had standardized a reflection type `std::meta::variable` in C++03 to represent what the standard called “variables” at the time. In C++11, the term “variable” was extended to include “references”. Such a change would have been difficult to do given that C++ by then likely would have had plenty of code that depended on a type arrangement around the more restricted definition of “variable”. That scenario is clearly backward-looking, but there is no reason to believe that similar changes might not be wanted in the future and we strongly believe that it behooves us to avoid adding undue constraints on the evolution of the language.

Other advantages of a single opaque type include:

- it makes no assumptions about the representation used within the implementation (e.g., it doesn’t advantage one compiler over another),
- it is trivially extensible (no types need to be added to represent additional language elements and meta-elements as the language evolves), and
- it allows convenient collections of heterogeneous constructs without having to surface reference semantics (e.g., a `std::vector<std::meta::info>` can easily represent a mixed template argument list — containing types and nontypes — without fear of slicing values).

§ 2 Examples

We start with a number of examples that show off what is possible with the proposed set of features. It is expected that these are mostly self-explanatory. Read ahead to the next sections for a more systematic description of each element of this proposal.

A number of our examples here show a few other language features that we hope to progress at the same time. This facility does not strictly rely on these features, and it is possible to do without them - but it would greatly help the usability experience if those could be adopted as well:

- expansion statements [\[P1306R1\]](#)
- non-transient `constexpr` allocation [\[P0784R7\]](#) [\[P1974R0\]](#) [\[P2670R1\]](#)

§ 2.1 Back-And-Forth

Our first example is not meant to be compelling but to show how to go back and forth between the reflection domain and the grammatical domain:

```
constexpr auto r = ^int;
typename[:r:] x = 42;      // Same as: int x = 42;
typename[:^char:] c = '*'; // Same as: char c = '*';
```

The `typename` prefix can be omitted in the same contexts as with dependent qualified names. For example:

```
using MyType = [:sizeof(int)<sizeof(long)? ^long : ^int:]; // Implicit
                "typename" prefix.
```

§ 2.2 Selecting Members

Our second example enables selecting a member “by number” for a specific type. It also shows the use of a metafunction dealing with diagnostics:

```
struct S { unsigned i:2, j:6; };

constexpr auto member_number(int n) {
    if (n == 0) return ^S::i;
    else if (n == 1) return ^S::j;
    else return std::meta::invalid_reflection("Only field numbers 0 and 1
        permitted");
}

int main() {
    S s{0, 0};
    s[:member_number(1):] = 42; // Same as: s.j = 42;
    s[:member_number(5):] = 0;  // Error (likely with "Only field numbers 0
        and 1 permitted" in text).
}
```

This example also illustrates that bit fields are not beyond the reach of this proposal.

§ 2.3 List of Types to List of Sizes

Here, sizes will be a `std::array<std::size_t, 3>` initialized with `{sizeof(int), sizeof(float), sizeof(double)}`:

```
constexpr std::array types = {^int, ^float, ^double};
constexpr std::array sizes = []{
    std::array<std::size_t, types.size()> r;
    std::ranges::transform(types, r.begin(), std::meta::size_of);
    return r;
}();
```

Compare this to the following type-based approach, which produces the same array sizes:

```
template<class...> struct list {};

using types = list<int, float, double>;

constexpr auto sizes = []<template<class...> class L, class... T>(L<T...>) {
    return std::array<std::size_t, sizeof...(T)>{{ sizeof(T)... }};
}(types{});
```

§ 2.4 Implementing make_integer_sequence

We can provide a better implementation of `make_integer_sequence` than a hand-rolled approach using regular template metaprogramming (although standard libraries today rely on an intrinsic for this):

```
#include <utility>
#include <vector>

template<typename T>
constexpr std::meta::info make_integer_seq_refl(T N) {
    std::vector args{^T};
    for (T k = 0; k < N; ++k) {
        args.push_back(std::meta::reflect_value(k));
    }
    return substitute(^std::integer_sequence, args);
}

template<typename T, T N>
using make_integer_sequence = [:make_integer_seq_refl<T>(N):];
```

§ 2.5 Getting Class Layout

```
struct member_descriptor
{
    std::size_t offset;
    std::size_t size;
};

// returns std::array<member_descriptor, N>
template <typename S>
constexpr auto get_layout() {
    constexpr auto members = nonstatic_data_members_of(^S);
    std::array<member_descriptor, members.size()> layout;
    for (int i = 0; i < members.size(); ++i) {
        layout[i] = {.offset=offset_of(members[i]),
                    .size=size_of(members[i])};
    }
    return layout;
}

struct X
{
    char a;
    int b;
    double c;
};
```

```

/*constexpr*/ auto Xd = get_layout<X>();

/*
where Xd would be std::array<member_descriptor, 3>{{
    { 0, 1 }, { 4, 4 }, { 8, 8 }
}}
*/

```

§ 2.6 Enum to String

One of the most commonly requested facilities is to convert an enum value to a string (this example relies on expansion statements):

```

template <typename E>
requires std::is_enum_v<E>
constexpr std::string enum_to_string(E value) {
    template for (constexpr auto e : std::meta::members_of(^E)) {
        if (value == [:e:]) {
            return std::string(std::meta::name_of(e));
        }
    }

    return "<unnamed>";
}

enum Color { red, green, blue };
static_assert(enum_to_string(Color::red) == "red");
static_assert(enum_to_string(Color(42)) == "<unnamed>");

```

We can also do the reverse in pretty much the same way:

```

template <typename E>
requires std::is_enum_v<E>
constexpr std::optional<E> string_to_enum(std::string_view name) {
    template for (constexpr auto e : std::meta::members_of(^E)) {
        if (name == std::meta::name_of(e)) {
            return [:e:];
        }
    }

    return std::nullopt;
}

```

But we don't have to use expansion statements - we can also use algorithms. For instance, `enum_to_string` can also be implemented this way (this example relies on non-transient `constexpr` allocation):

```

template <typename E>
requires std::is_enum_v<E>
constexpr std::string enum_to_string(E value) {
    constexpr auto enumerators =
        std::meta::members_of(^E)
        | std::views::transform([](std::meta::info e){

```

```

        return std::pair<E, std::string>(std::meta::value_of<E>(e),
                                         std::meta::name_of(e));
    })
    | std::ranges::to<std::map>();

    auto it = enumerators.find(value);
    if (it != enumerators.end()) {
        return it->second;
    } else {
        return "<unnamed>";
    }
}

```

Note that this last version has lower complexity: While the versions using an expansion statement use an expected $O(N)$ number of comparisons to find the matching entry, a `std::map` achieves the same with $O(\log(N))$ complexity (where N is the number of enumerator constants).

§ 2.7 Parsing Command-Line Options

Our next example shows how a command-line option parser could work by automatically inferring flags based on member names. A real command-line parser would of course be more complex, this is just the beginning:

```

template<typename Opts>
auto parse_options(std::span<std::string_view const> args) -> Opts {
    Opts opts;
    template for (constexpr auto dm : nonstatic_data_members_of(^Opts)) {
        auto it = std::ranges::find_if(args,
            [](std::string_view arg){
                return arg.starts_with("--") && arg.substr(2) == name_of(dm);
            });

        if (it == args.end()) {
            // no option provided, use default
            continue;
        } else if (it + 1 == args.end()) {
            std::print(stderr, "Option {} is missing a value\n", *it);
            std::exit(EXIT_FAILURE);
        }

        using T = typename[:type_of(dm):];
        auto iss = ispanstream(it[1]);
        if (iss >> opts[:dm:]; !iss) {
            std::print(stderr, "Failed to parse option {} into a {}\n", *it,
                display_name_of(^T));
            std::exit(EXIT_FAILURE);
        }
    }
    return opts;
}

struct MyOpts {
    string file_name = "input.txt"; // Option "--file_name <string>"
    int count = 1; // Option "--count <int>"
}

```

```
};

int main(int argc, char *argv[]) {
    MyOpts opts = parse_options<MyOpts>(std::vector<std::string_view>(argv+1,
        argv+argc));
    // ...
}
```

(This example is based on a presentation by Matúš Chochlík.)

§ 2.8 A Simple Tuple Type

```
#include <meta>

template<typename... Ts> struct Tuple {
    using storage =
        typename[:std::meta::synth_struct({nsdm_description(^T)...}):];
    storage data;

    Tuple(): data{} {}
    Tuple(Ts const& ...vs): data{ vs... } {}
};

template<typename... Ts>
struct std::tuple_size<Tuple<Ts...>>: public integral_constant<size_t,
    sizeof...(Ts)> {};

template<typename I, typename... Ts>
struct std::tuple_element<I, Tuple<Ts...>> {
    using type = [: template_arguments_of(^Tuple<Ts...>)[I] :];
};

template<typename I, typename... Ts>
constexpr auto get(Tuple<Ts...> &t) noexcept -> std::tuple_element_t<I,
    Tuple<Ts...>& {
    return t.data.[:nonstatic_data_members_of(^decltype(t.data))[I]:];
}

// Similarly for other value categories...
```

This example uses a “magic” `std::meta::synth_struct` template along with member reflection through the `members_of` metafunction to implement a `std::tuple`-like type without the usual complex and costly template metaprogramming tricks that that involves when these facilities are not available.

§ 2.9 Struct to Struct of Arrays

```
constexpr auto make_struct_of_arrays(std::meta::info type, size_t n) ->
    std::meta::info {
    std::vector<info> new_members;
    for (std::meta::info member : nonstatic_data_members_of(type)) {
        auto array_type = substitute(^std::array, {type_of(member),
            reflect_value(n)});
        new_members.push_back(nsdm_description(array_type, {.name =
            name_of(member)}));
    }
```



```

    }
    return std::meta::synth_struct(new_members);
}

template <typename T, size_t N>
using struct_of_arrays = [: make_struct_of_arrays(^T, N) :];

```

Example:

```

struct point {
    float x;
    float y;
    float z;
};

using points = struct_of_arrays<point, 30>;
// equivalent to:
// struct points {
//     std::array<float, 30> x;
//     std::array<float, 30> y;
//     std::array<float, 30> z;
// };

```

§ 2.10 Parsing Command-Line Options II

Now that we've seen a couple examples of using `std::meta::synth_struct` to create a type, we can create a more sophisticated command-line parser example. This isn't a complete implementation, but hopefully is enough to demonstrate the utility. This is the opening example for [clap](#) (Rust's Command Line Argument Parser):

```

struct Args : Clap {
    Option<std::string, {.use_short=true, .use_long=true}> name;
    Option<int, {.use_short=true, .use_long=true}> count = 1;
};

int main(int argc, char** argv) {
    auto opts = Args{}.parse(argc, argv);

    for (int i = 0; i < opts.count; ++i) { // opts.count has type int
        std::print("Hello {}!", opts.name); // opts.name has type std::string
    }
}

```

Which we can implement like this:

```

struct Flags {
    bool use_short;
    bool use_long;
};

// type that has a member optional<T> with some suitable constructors and
members
template <typename T, Flags flags>
struct Option;

```

```

// convert a type (all of whose non-static data members are specializations
// of Option)
// to a type that is just the appropriate members.
// For example, if type is a reflection of the Args presented above, then
// this
// function would evaluate to a reflection of the type
// struct {
//     std::string name;
//     int count;
// }
constexpr auto spec_to_opts(std::meta::info type) -> std::meta::info {
    std::vector<std::meta::info> new_members;
    for (std::meta::info member : nonstatic_data_members_of(type)) {
        auto new_type = template_arguments_of(type_of(member))[0];
        new_members.push_back(nsdm_description(new_type,
            {.name=name_of(member)}));
    }
    return std::meta::synth_struct(new_members);
}

struct Clap {
    template <typename Spec>
    auto parse(this Spec const& spec, int argc, char** argv) {
        std::vector<std::string_view> cmdline(argv+1, argv+argc)

        // check if cmdline contains --help, etc.

        using Opts = [: spec_to_opts(^Spec) :];
        Opts opts;

        template for (constexpr auto [sm, om] :
            std::views::zip(nonstatic_data_members_of(^Spec),
                nonstatic_data_members_of(^Opts))) {
            auto const& cur = spec.[:sm:];
            constexpr auto type = type_of(om);

            // find the argument associated with this option
            auto it = std::ranges::find_if(cmdline,
                [&](std::string_view arg){
                    return (cur.use_short && arg.starts_with("-") && arg.substr(1) ==
                        name_of(sm))
                        || (cur.use_long && arg.starts_with("--") && arg.substr(2) ==
                        name_of(sm));
                });

            // no such argument
            if (it == cmdline.end()) {
                if constexpr (template_of(om) == ^std::optional) {
                    // the type is optional, so the argument is too
                    continue;
                } else if (cur.initializer) {
                    // the type isn't optional, but an initializer is provided, use
                    that
                    opts.[:om:] = *cur.initializer;
                    continue;
                }
            }
        }
    }
};

```

```

    } else {
        std::print(stderr, "Missing required option {}\n", name_of(sm));
        std::exit(EXIT_FAILURE);
    }
} else if (it + 1 == cmdline.end()) {
    std::print(stderr, "Option {} for {} is missing a value\n", *it,
        name_of(sm));
    std::exit(EXIT_FAILURE);
}

// found our argument, try to parse it
auto iss = ispanstream(it[1]);
if (iss >> opts[:om:]; !iss) {
    std::print(stderr, "Failed to parse {:?} into option {} of type
        {}\n",
        *it, name_of(sm), display_name_of(type));
    std::exit(EXIT_FAILURE);
}
}
return opts;
}
}

```

§ 2.11 A Universal Formatter

This example is taken from Boost.Describe:

```

struct universal_formatter {
    constexpr auto parse(auto& ctx) { return ctx.begin(); }

    template <typename T>
    auto format(T const& t, auto& ctx) const {
        auto out = std::format_to(ctx.out(), "{}{{", name_of(^T));

        auto delim = [first=true]() mutable {
            if (!first) {
                *out++ = ',';
                *out++ = ' ';
            }
            first = false;
        };

        template for (constexpr auto base : bases_of(^T)) {
            delim();
            out = std::format_to(out, "{}", static_cast<[:base:] const&>(t));
        }

        template for (constexpr auto mem : nonstatic_data_members_of(^T)) {
            delim();
            out = std::format_to(out, ".{}={}", name_of(mem), t.[:mem:]);
        }

        *out++ = '}' ;
        return out;
    }
}

```

```
};

struct X { int m1 = 1; };
struct Y { int m2 = 2; };
class Z : public X, private Y { int m3 = 3; int m4 = 4; };

template <> struct std::formatter<X> : universal_formatter { };
template <> struct std::formatter<Y> : universal_formatter { };
template <> struct std::formatter<Z> : universal_formatter { };

int main() {
    std::println("{} ", Z()); // Z{X{.m1 = 1}, Y{.m2 = 2}, .m3 = 3, .m4 = 4}
}
```

§ 2.12 Implementing member-wise hash_append

Based on the [\[N3980\]](#) API:

```
template <typename H, typename T> requires std::is_standard_layout_v<T>
void hash_append(H& algo, T const& t) {
    template for (constexpr auto mem : nonstatic_data_members_of(^T)) {
        hash_append(algo, t.[:mem:]);
    }
}
```

§ 2.13 Converting a Struct to a Tuple

This approach requires allowing packs in structured bindings [\[P1061R5\]](#), but can also be written using `std::make_index_sequence`:

```
template <typename T>
constexpr auto struct_to_tuple(T const& t) {
    constexpr auto members = nonstatic_data_members_of(^T);

    constexpr auto indices = []{
        std::array<int, members.size()> indices;
        std::ranges::iota(indices, 0);
        return indices;
    }();

    constexpr auto [...Is] = indices;
    return std::make_tuple(t.[: members[Is] :]...);
}
```

An alternative approach is:

```
constexpr auto struct_to_tuple_type(info type) -> info {
    return substitute(^std::tuple,
        nonstatic_data_members_of(type)
        | std::ranges::transform(std::meta::type_of)
        | std::ranges::transform(std::meta::remove_cvref)
        | std::ranges::to<std::vector>());
}
```

```

template <typename To, typename From, std::meta::info ... members>
constexpr auto struct_to_tuple_helper(From const& from) -> To {
    return To(from.[:members:]...);
}

template <typename From>
constexpr auto struct_to_tuple(From const& from) {
    using To = [: struct_to_tuple_type(^From): ];

    std::vector args = {^To, ^From};
    for (auto mem : nonstatic_data_members_of(^From)) {
        args.push_back(reflect_value(mem));
    }

    auto f = entity_ref<To(From const&)>(substitute(^struct_to_tuple_helper,
        args));
    return f(from);
}

```

Here, `struct_to_tuple_type` takes a reflection of a type like `struct { T t; U const& u; V v; }` and returns a reflection of the type `std::tuple<T, U, V>`. `helper`. That gives us the return type. Then, `struct_to_tuple_helper` is a function template that does the actual conversion - which it can do by having all the reflections of the members as a non-type template parameter pack.

Everything is put together by using `substitute` to create the instantiation of `struct_to_tuple_helper` that we need, which is use `entity_ref` to get the correct function out of. `f` there is a function pointer to the correct specialization of `struct_to_tuple_helper`. Which we can simply invoke.

§ 3 Proposed Features

§ 3.1 The Reflection Operator (^)

The reflection operator produces a reflection value from a grammatical construct (its operand):

unary-expression:

```

...
^ ::
^ namespace-name
^ type-id
^ cast-expression

```

Note that *cast-expression* includes *id-expression*, which in turn can designate templates, member names, etc.

The current proposal requires that the *cast-expression* be:

- a *primary-expression* referring to a function or member function, or
- a *primary-expression* referring to a variable, static data member, or structured binding, or
- a *primary-expression* referring to a nonstatic data member, or

- a *primary-expression* referring to a template, or
- a constant-expression.

In a SFINAE context, a failure to substitute the operand of a reflection operator construct causes that construct to evaluate to an invalid reflection.

§ 3.2 Splicers (`[ : ... : ]`)

A reflection that is not an invalid reflection can be “spliced” into source code using one of several *splicer* forms:

- `[ : r : ]` produces an *expression* evaluating to the entity or constant value represented by `r`.
- `typename[ : r : ]` produces a *simple-type-specifier* corresponding to the type represented by `r`.
- `template[ : r : ]` produces a *template-name* corresponding to the template represented by `r`.
- `namespace[ : r : ]` produces a *namespace-name* corresponding to the namespace represented by `r`.
- `[ : r : ] ::` produces a *nested-name-specifier* corresponding to the namespace, enumeration type, or class type represented by `r`.

Attempting to splice a reflection value that does not meet the requirement of the splice (including “invalid reflections”) is ill-formed. For example:

```
typename[ : ^:: : ] x = 0; // Error.
```

A quality implementation should emit the diagnostic text associated with an invalid reflection when attempting to splice that invalid reflection.

(This proposal does not at this time propose range-based splicers as described in P1240. We still believe that those are desirable. However, they are more complex to implement and they involve syntactic choices that benefit from being considered along with other proposals that introduce pack-like constructs in non-template contexts. Meanwhile, we found that many very useful techniques are enabled with just the basic splicers presented here.)

§ 3.2.1 Range Splicers

The splicers described above all take a single object of type `std::meta::info` (described in more detail below). However, there are many cases where we don’t have a single reflection, we have a range of reflections - and we want to splice them all in one go. For that, we need a different form of splicer: a range splicer.

Construct the [struct-to-tuple](#) example from above. It was demonstrates using a single splice, but it would be simpler if we had a range splice:

With Single Splice	With Range Splice
<pre> template <typename T> constexpr auto struct_to_tuple(T const& t) { constexpr auto members = nonstatic_data_members_of(^T); constexpr auto indices = []{ std::array<int, members.size()> indices; std::ranges::iota(indices, 0); return indices; }(); constexpr auto [...Is] = indices; return std::make_tuple(t.[: members[Is] :]...); } </pre>	<pre> template <typename T> constexpr auto struct_to_tuple(T const& t) { constexpr auto members = nonstatic_data_members_of(^T); return std::make_tuple(t.[: ...members :]...); } </pre>

A range splice, `[ : ... r : ]`, would accept as its argument a constant range of `meta::info`, `r`, and would behave as an unexpanded pack of splices. So the above expression

```
make_tuple(t.[: ... members :]...)
```

would evaluate as

```
make_tuple(t.[:members[0]:], t.[:members[1]:], ..., t.[:members[N-1]:])
```

This is a very useful facility indeed!

However, range splicing of dependent arguments is at least an order of magnitude harder to implement than ordinary splicing. We think that not including range splicing gives us a better chance of having reflection in C++26. Especially since, as this paper’s examples demonstrate, a lot can be done without them.

Another way to work around a lack of range splicing would be to implement `with_size<N>(f)`, which would behave like `f(integral_constant<size_t, 0>{}, integral_constant<size_t, 0>{}, ..., integral_constant<size_t, N-1>{})`. Which is enough for a tolerable implementation:

```

template <typename T>
constexpr auto struct_to_tuple(T const& t) {
    constexpr auto members = nonstatic_data_members_of(^T);
    return with_size<members.size()>([&](auto... Is){
        return std::make_tuple(t.[: members[Is] :]...);
    });
}

```

§ 3.3 `std::meta::info`

The type `std::meta::info` can be defined as follows:

```
namespace std {
    namespace meta {
        using info = decltype(^int);
    }
}
```

In our initial proposal a value of type `std::meta::info` can represent:

- an error (corresponding to an “invalid reflection”)
- any (C++) type and type-alias
- any function or member function
- any variable, static data member, or structured binding
- any non-static data member
- any constant value
- any template
- any namespace

Notably absent at this time are general non-constant expressions (that aren’t *expression-ids* referring to variables or structured bindings). For example:

```
int x = 0;
void g() {
    [:^(x):] = 42;    // Okay. Same as: x = 42;
    x = [:^(2*x):];  // Error: "2*x" is a general non-constant expression.
    constexpr int N = 42;
    x = [:^(2*N):];  // Okay: "2*N" is a constant-expression.
}
```

Note that for `:^(2*N)` an implementation only has to capture the constant value of `2*N` and not various other properties of the underlying expression (such as any temporaries it involves, etc.).

The type `std::meta::info` is a *scalar* type. Nontype template arguments of type `std::meta::info` are permitted. The entity being reflected can affect the linkage of a template instance involving a reflection. For example:

```
template<auto R> struct S {};

extern int x;
static int y;

S<^x> sx; // S<^x> has external name linkage.
S<^y> sy; // S<^y> has internal name linkage.
```

Namespace `std::meta` is associated with type `std::meta::info`. That allows the core meta functions to be invoked without explicit qualification. For example:

```
#include <meta>
struct S {};
```



```
std::string name2 = std::meta::name_of(^S); // Okay.
std::string name1 = name_of(^S);           // Also okay.
```

§ 3.4 Metafunctions

We propose a number of metafunctions declared in namespace `std::meta` to operator on reflection values. Adding metafunctions to an implementation is expected to be relatively “easy” compared to implementing the core language features described previously. However, despite offering a normal consteval C++ function interface, each on of these relies on “compiler magic” to a significant extent.

§ 3.4.1 `invalid_reflection`, `is_invalid`, `diagnose_error`

```
namespace std::meta {
    consteval auto invalid_reflection(string_view msg, source_location loc =
        source_location::current()) -> info;
    consteval auto is_invalid(info) -> bool;
    consteval auto diagnose_error(info) -> void;
}
```

An invalid reflection represents a potential diagnostic for an erroneous construct. Some standard metafunctions will generate such invalid reflections, but user programs can also create them with the `invalid_reflection` metafunction. `is_invalid` returns true if it is given an invalid reflection. Evaluating `diagnose_error` renders a program ill-formed. If the given reflection is for an invalid reflection, an implementation is encouraged to render the encapsulated message and source position as part of the diagnostic indicating that the program is ill-formed.

§ 3.4.2 `name_of`, `display_name_of`, `source_location_of`

```
namespace std::meta {
    consteval auto name_of(info r) -> string_view;
    consteval auto display_name_of(info r) -> string_view;
}
```

Given a reflection `r` that designates a declared entity `x`, `name_of(r)` returns a `string_view` holding the unqualified name of `x`. For all other reflections, an empty `string_view` is produced. For template instances, the name does not include the template argument list. The contents of the `string_view` consist of characters of the basic source character set only (an implementation can map other characters using universal character names).

Given a reflection `r`, `display_name_of(r)` returns an unspecified non-empty `string_view`. Implementations are encouraged to produce text that is helpful in identifying the reflected construct.

Given a reflection `r`, `source_location_of(r)` returns an unspecified `source_location`. Implementations are encouraged to produce the correct source location of the item designated by the reflection.

§ 3.4.3 `type_of`, `parent_of`, `entity_of`

```
namespace std::meta {
    consteval auto type_of(info r) -> info;
```

```

consteval auto parent_of(info r) -> info;
consteval auto entity_of(info r) -> info;
}

```

If `r` is a reflection designating a typed entity, `type_of(r)` is a reflection designating its type. Otherwise, `type_of(r)` produces an invalid reflection.

If `r` designates a member of a class or namespace, `parent_of(r)` is a reflection designating its immediately enclosing class or namespace. Otherwise, `parent_of(r)` produces an invalid reflection.

If `r` designates an alias, `entity_of(r)` designates the underlying entity. Otherwise, `entity_of(r)` produces `r`.

§ 3.4.4 `template_of`, `template_arguments_of`

```

namespace std::meta {
    consteval auto template_of(info r) -> info;
    consteval auto template_arguments_of(info r) -> vector<info>;
}

```

If `r` is a reflection designated a type that is a specialization of some template, then `template_of(r)` is a reflection of that template and `template_arguments_of(r)` is a vector of the reflections of the template arguments. Otherwise, both yield invalid reflections. In other words, the preconditions on both is that `has_template_arguments(r)` is `true`.

For example:

```

std::vector<int> v = {1, 2, 3};
static_assert(template_of(type_of(^v)) == ^std::vector);
static_assert(template_arguments_of(type_of(^v))[0] == ^int);

```

§ 3.4.5 `members_of`, `nonstatic_data_members_of`, `bases_of`, `enumerators_of`, `subobjects_of`

```

namespace std::meta {
    template<typename ...Fs>
        consteval auto members_of(info class_type, Fs ...filters) ->
            vector<info>;
    template<typename ...Fs>
        consteval auto nonstatic_data_members_of(info class_type, Fs ...filters)
            -> vector<info> {
            return members_of(class_type, is_nonstatic_data_member, filters...);
        }
    template<typename ...Fs>
        consteval auto bases_of(info class_type, Fs ...filters) -> vector<info>
        {
            return members_of(class_type, is_base, filters...);
        }
    template<typename ...Fs>
        consteval auto enumerators_of(info class_type, Fs ...filters) ->
            vector<info>;
    template<typename ...Fs>

```

```

consteval auto subobjects_of(info class_type, Fs ...filters) ->
    vector<info>;
}

```

§ 3.4.6 substitute

```

namespace std::meta {
    consteval auto substitute(info templ, span<info const> args) -> info;
}

```

Given a reflection for a template and reflections for template arguments that match that template, `substitute` returns a reflection for the entity obtained by substituting the given arguments in the template.

For example:

```

constexpr auto r = substitute(^std::vector, std::vector{^int});
using T = [:r:]; // Ok, T is std::vector<int>

```

This process might kick off instantiations outside the immediate context, which can lead to the program being ill-formed. Substitution errors in the immediate context of the template result in an invalid reflection being returned.

Note that the template is only substituted, not instantiated. For example:

```

template<typename T> struct S { typename T::X x; };

constexpr auto r = substitute(^S, std::vector{^int}); // Okay.
typename[:r:] si; // Error: T::X is invalid for T = int.

```

§ 3.4.7 entity_ref<T>, value_of<T>, ptr_to_member<T>

```

namespace std::meta {
    template<typename T> consteval auto entity_ref(info var_or_func) -> T&;
    template<typename T> consteval auto value_of(info constant_expr) -> T;
    template<typename T> consteval auto pointer_to_member(info member) -> T;
}

```

If `r` is a reflection of a variable or function of type `T`, `entity_ref<T>(r)` evaluates to a reference to that variable or function. Otherwise, `entity_ref<T>(r)` is ill-formed.

If `r` is a reflection for a constant-expression or a constant-valued entity of type `T`, `value_of(r)` evaluates to that constant value. Otherwise, `value_of<T>(r)` is ill-formed.

If `r` is a reflection for a non-static member or for a constant pointer-to-member value matching type `T`, `pointer_to_member<T>(r)` evaluates to a corresponding pointer-to-member value. Otherwise, `value_of<T>(r)` is ill-formed.

These functions may feel similar to splicers, but unlike splicers they do not require their operand to be a constant-expression itself. Also unlike splicers, they require knowledge of the type associated with the entity reflected by their operand.

§ 3.4.8 test_type<Pred>

```

namespace std::meta {
    auto test_type(info templ, info type) -> bool {
        return test_types(templ, vector{type});
    }

    auto test_types(info templ, span<info const> types) -> bool {
        return value_of<bool>(substitute(templ, types));
    }
}

```

This utility translates existing metaprogramming predicates (expressed as constexpr variable templates) to the reflection domain. For example:

```

struct S {};
static_assert(test_type(^std::is_class_v, ^S));

```

An implementation is permitted to recognize standard predicate templates and implement test_type without actually instantiating the predicate template. In fact, that is recommended practice.

§ 3.4.9 Other Singular Reflection Predicates

```

namespace std::meta {
    constexpr auto is_public(info r) -> bool;
    constexpr auto is_protected(info r) -> bool;
    constexpr auto is_private(info r) -> bool;
    constexpr auto is_accessible(info r) -> bool;
    constexpr auto is_virtual(info r) -> bool;
    constexpr auto is_deleted(info entity) -> bool;
    constexpr auto is_defaulted(info entity) -> bool;
    constexpr auto is_explicit(info entity) -> bool;
    constexpr auto is_override(info entity) -> bool;
    constexpr auto is_pure_virtual(info entity) -> bool;
    constexpr auto has_static_storage_duration(info r) -> bool;

    constexpr auto is_nsdm(info entity) -> bool;
    constexpr auto is_base(info entity) -> bool;
    constexpr auto is_namespace(info entity) -> bool;
    constexpr auto is_function(info entity) -> bool;
    constexpr auto is_static(info entity) -> bool;
    constexpr auto is_variable(info entity) -> bool;
    constexpr auto is_type(info entity) -> bool;
    constexpr auto is_alias(info entity) -> bool;
    constexpr auto is_incomplete_type(info entity) -> bool;
    constexpr auto is_template(info entity) -> bool;
    constexpr auto is_function_template(info entity) -> bool;
    constexpr auto is_variable_template(info entity) -> bool;
    constexpr auto is_class_template(info entity) -> bool;
    constexpr auto is_alias_template(info entity) -> bool;
    constexpr auto has_template_arguments(info r) -> bool;
}

```

§ 3.4.10 reflect_value

```
namespace std::meta {
    template<typename T> consteval auto reflect_value(T value) -> info;
}
```

This metafunction produces a reflection representing the constant value of the operand.

§ 3.4.11 nsdm_description, synth_struct, synth_union

```
namespace std::meta {
    struct nsdm_field_args {
        optional<string_view> name;
        optional<int> alignment;
        optional<int> width;
    };

    consteval auto nsdm_description(info type, nsdm_field_args args = {}) ->
        info;

    consteval auto synth_struct(span<info const>) -> info;
    consteval auto synth_union(span<info const>) -> info;
}
```

`nsdm_description` creates a reflection that describes a non-static data member of given type. Optional alignment, bit-field-width, and name can be provided as well. If type does not designate a valid data member type, an invalid reflection is produced. If no name is provided, the name of the non-static data member is unspecified. Note that the reflection obtained from `nsdm_description` is *not* the reflection of a non-static data member itself; it only encapsulates the information needed to synthesize such a data member. In particular, metafunctions like `name_of`, `type_of`, and `parent_of` are not applicable to the result of an `nsdm_description`.

`synth_struct` and `synth_union` take a range of NSDM descriptions and return a reflection that denotes a struct and union, respectively, comprised of corresponding non-static data members.

For example:

```
constexpr auto T = std::meta::synth_struct({
    nsdm_description(^int),
    nsdm_description(^char),
    nsdm_description(^double),
});

// T is a reflection of the type
// struct {
//     int _0;
//     char _1;
//     double _2;
// }

constexpr auto U = std::meta::synth_struct({
    nsdm_description(^int, {.name="i", .align=64}),
    nsdm_description(^int, {.name="j", .align=64}),
});
```

```
// U is a reflection of the type
// struct {
//     alignas(64) int i;
//     alignas(64) int j;
// }
```

§ 3.4.12 Data Layout Reflection

```
namespace std::meta {
    constexpr auto offset_of(info entity) -> size_t;
    constexpr auto size_of(info entity) -> size_t;

    constexpr auto bit_offset_of(info entity) -> size_t;
    constexpr auto bit_size_of(info entity) -> size_t;
}
```

§ 4 References

[N3980] H. Hinnant, V. Falco, J. Byteway. 2014-05-24. Types don't know #.

<https://wg21.link/n3980>

[P0784R7] Daveed Vandevoorde, Peter Dimov, Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More constexpr containers.

<https://wg21.link/p0784r7>

[P1061R5] Barry Revzin, Jonathan Wakely. 2023-05-18. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r5>

[P1240R2] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. 2022-01-14. Scalable Reflection.

<https://wg21.link/p1240r2>

[P1306R1] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2019-01-21. Expansion statements.

<https://wg21.link/p1306r1>

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient constexpr allocation using propconst.

<https://wg21.link/p1974r0>

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.

<https://wg21.link/p2237r0>

[P2670R1] Barry Revzin. 2023-02-03. Non-transient constexpr allocation.

<https://wg21.link/p2670r1>